UI.py

Student worksheet - build the console user interface

Create a file called ui.py. Work through the steps in order. Each part adds one small piece to the final program. The final version should be able to display information to the user and receive input from the console.

This worksheet does not assume you have implemented cards.py or player.py before. You can treat Card and Player as placeholder objects.

Before you start: Use meaningful names and keep the code readable. Test after each section instead of writing the whole file first.

0. Set up the file

At the top of ui.py, add the imports you will need.

```
from cards  import Card
from player import Player
```

Hint: This file is primarily concerned with printing information and reading input from the console

1. create the method show_table(self, community_cards, pot):

This method should display the community cards and the current pot:

- format the community cards into a string
- if no cards are present yet, display '(empty)'
- print the board and the pot

```
def show_table(self, community_cards: list[Card], pot: int) -> None:
    ...
```

Checkpoint:

Expected output format:

```
Board: AH KD 7S
Pot: 120
```

2. create the method show_player(self, player):

Display the following information about the player:

- Name
- Hole cards
- Remaining chips

```
def show_player(self, player: Player) -> None:
    ...
```

3. create the method ask_action(self, player, call_amount):

This method should prompt the player to choose an action.

Accepted inputs and their corresponding return values:

- c, call, or check → return 'call'
- r or raise → return 'raise'
- f or fold → return 'fold'

```
def ask_action(self, player: Player, call_amount: int) -> str:
    ...
```

Hint: Use a while True loop so that invalid input causes the prompt to repeat until a valid choice is made.

```
action = input(...).strip().lower()
if action in {'c', 'call', 'check'}:
    return 'call'
```

Checkpoint:

Confirm correct normalization:

```
# If the user types 'R', the method should still return 'raise'.
```

4. create the method ask_raise_amount(self, minimum, maximum):

This method should ask the user how much they want to raise:

- ask for input
- convert the value to int
- handle non-numeric input using try/except
- ensure the value is between minimum and maximum (inclusive)

```
try:
    amount = int(raw)
except ValueError:
```

```
    . . .
```

Hint: continue restarts the loop after invalid input, preventing the function from returning a bad value.

5. create the method show_message(self, message):

This method should simply print a message to the console.

6. create the method format_cards(self, cards):

This method should return all cards formatted as a single string. Example output: AH KD 7S

Hint: join() combines all items in an iterable into a single string, inserting a space between each card.

Common mistakes to check

- Forgetting .lower() when comparing user input.
- Not handling non-integer input with try/except.
- Returning the wrong action string (e.g., returning 'c' instead of 'call').
- Forgetting to put spaces between card strings.
- Not using a loop for repeated input, causing the function to exit on invalid input.